

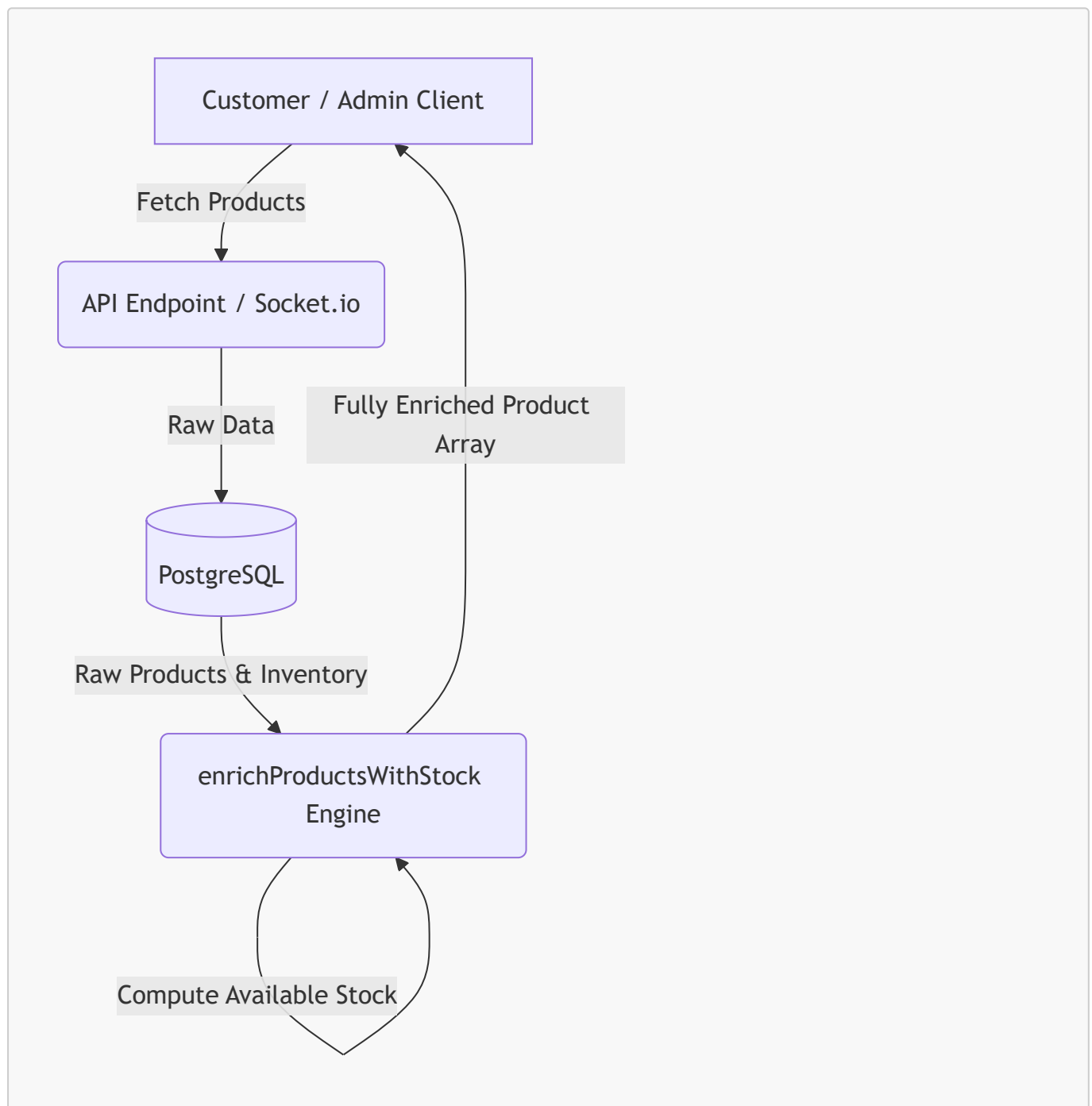
Inventory System Capabilities & Concurrency Documentation

This document describes the architectural, functional, and capability changes implemented in the inventory and ordering subsystems of the Capstone project.

1. Core System Architecture

Dynamic Query-Time Resolution & Locked Stock Prevention

To solve the issue of stuck/locked stock (when database columns drift), the system calculates product reservations **on-the-fly** at query-time.

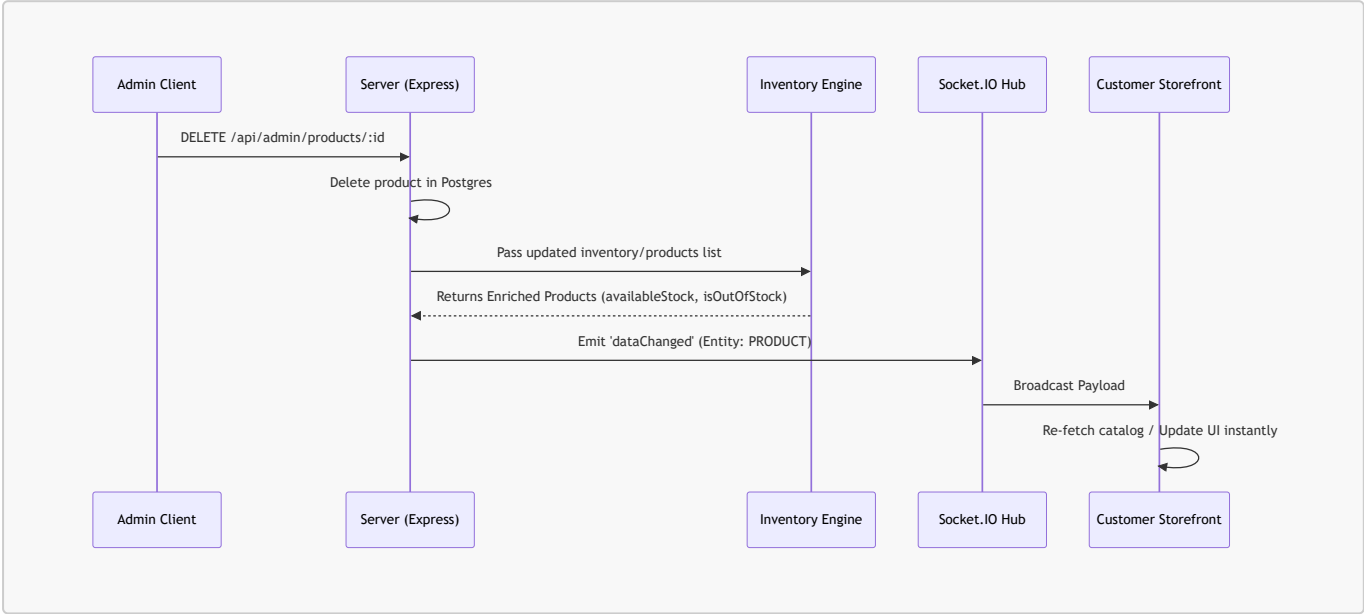


Self-Healing Database Reconciliation

We maintain a self-healing function `reconcileReservedCounts()` that ensures the database columns remain synchronized and heals them automatically on server startup and during transactional commits.

Automated Socket.IO & API Enrichment

Every payload emitted over Socket.IO (updates, creations, deletions) is intercepted and enriched with real-time stock limits before transmission.



SSR React Hydration Protection

Because the customer storefront is Server-Side Rendered (SSR) by Next.js, reading dynamic client-side `localStorage` data (like basket quantity or user wallet balances) during the initial page load causes React hydration crashes.

[!IMPORTANT] **Client Mounting Checks:** Dynamic components (RightPanel, StorefrontHeader, DashboardSidebar) implement a rigorous `mounted` state check. They intentionally render safe "empty" defaults (`0 Items`, empty lists) on the server, and hydrate with the live `Zustand` store data once safely mounted in the user's browser.

2. Modules & Functions Overview

Module / Component	Function / Endpoint	Purpose & Behavior
<code>inventoryManager.js</code>	<code>enrichProductsWithStock(products, tx)</code>	Calculates real-time stock limits. Resolves blanks vs recipe limits dynamically. Returns <code>availableStock</code> and <code>isOutOfStock</code> .

Module / Component	Function / Endpoint	Purpose & Behavior
<code>inventoryManager.js</code>	<code>reconcileReservedCounts(tx)</code>	Scans all active orders and repairs/syncs the <code>reservedCount</code> DB columns.
<code>socketUtil.js</code>	<code>emitDataChanged(...)</code>	Enriches product payloads asynchronously before transmitting data to socket clients.
<code>customerController.js</code>	<code>submitOrder(req, res)</code>	Enforces database locks using <code>SELECT ... FOR UPDATE</code> on blanks and spools. Throws errors and rolls back on failure.
<code>frontend/app/page.tsx</code>	<code>useEffect</code> Socket Listener	Listens for <code>PRODUCT</code> , <code>ORDER</code> , and <code>INVENTORY</code> emissions and re-fetches products to guarantee real-time catalog accuracy.

3. Concurrency & Ordering Situations

The system is rigorously tested against edge cases. Below is a detailed breakdown of how specific transactional situations are mitigated.

Scenario	Initial State	System Action	Outcome
A. Concurrent Race Condition	Spool Count = 50. Order A and Order B simultaneously try to checkout 50 items.	Order A locks the row using <code>FOR UPDATE</code> . Order B blocks. Order A commits and updates the DB. Order B unblocks, reads the new empty stock, and Rolls Back.	Order A succeeds. Order B fails gracefully with <code>OutOfStock</code> . No negative stock.
B. UI Quantity Block	Ryven stock is 2. Customer clicks <code>+</code> to add a 3rd Ryven.	The frontend interceptor compares new quantity (3) against <code>availableStock</code> (2). It immediately blocks the action.	Basket stays at 2. User sees an error toast instantly.
C. Mixed Checkout Failure	Basket has 2 in-stock items and 1 out-of-stock item. Customer clicks checkout.	Backend checks Item A (OK). Checks Item B (Fails). The Prisma Transaction throws an exception and entirely rolls back.	Order fails entirely. No DB updates, no wallet deductions.
D. Manual Database Drift	Admin deletes an order in DB manually, leaving a stranded <code>reservedCount = 10</code> .	Server dynamically calculates active orders (<code>0</code>), bypassing the corrupted <code>10</code> . <code>reconcileReservedCounts()</code> runs on the next request.	Customer views correct stock. DB heals its broken column.

Scenario	Initial State	System Action	Outcome
E. Raw Material Deletion	"Gold Jacket" requires "Gold Thread". Admin deletes "Gold Thread" entirely.	<code>enrichProductsWithStock</code> recalculates. Required material evaluates to 0. Available stock drops to 0. Socket syncs to customers.	Jacket drops to 0 stock instantly. Ordering is prevented.

Visualizing the Race Condition (Situation A)

